

# Data Structures and Algorithms

GÉZA HORVÁTH

Fourth Lecture

# Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 **Elementary data structures: stacks, queues, lists.**
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

# The stack – abstract data type

In computer science, a stack is an abstract data type that serves as a collection of elements, with two principal operations:

- PUSH, which **adds** an element to the collection, and
- POP, which **accesses** and **removes** the most recently added element that was not yet removed.

The order in which elements come off a stack gives rise to its alternative name, **LIFO** (last in, first out).

See the drawing on the whiteboard!

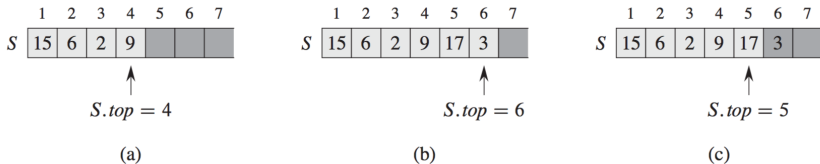
# The stack – data structure

Stack properties:

- solid
- sequential
- dynamic
- continuously represented

# The stack – representation

Let  $S$  be a stack containing numbers. We can implement a stack of at most  $n$  elements with an array  $S[1, \dots, n]$ . The array has an attribute  **$S.top$**  that indexes the most recently inserted element.



**Figure 10.1** An array implementation of a stack  $S$ . Stack elements appear only in the lightly shaded positions. (a) Stack  $S$  has 4 elements. The top element is 9. (b) Stack  $S$  after the calls  $PUSH(S, 17)$  and  $PUSH(S, 3)$ . (c) Stack  $S$  after the call  $POP(S)$  has returned the element 3, which is the one most recently pushed. Although element 3 still appears in the array, it is no longer in the stack; the top is element 17.

# The stack – representation

When  $S.top=0$ , the stack contains no elements and is empty. We can test to see whether the stack is empty by query operation `STACK-EMPTY`.

- If we attempt to pop an empty stack, we say the stack **underflows**, which is normally an **error**.
- If  $S.top$  exceeds  $n$ , the stack **overflows**.

In our pseudocode implementation, we don't worry about stack overflow.

# The stack – operations

We can implement each of the stack operations with just a few lines of code.

STACK-EMPTY( $S$ )

```
1  if  $S.top == 0$ 
2      return TRUE
3  else return FALSE
```

PUSH( $S, x$ )

```
1   $S.top = S.top + 1$ 
2   $S[S.top] = x$ 
```

POP( $S$ )

```
1  if STACK-EMPTY( $S$ )
2      error “underflow”
3  else  $S.top = S.top - 1$ 
4      return  $S[S.top + 1]$ 
```

# The stack – data structure operations

## Implemented data structure operations:

- **create** data structure – continuous (vector) representation
- **modify** data structure
  - add elements – PUSH
  - delete elements – POP
  - change elements – **not possible**
- **access** – POP (and you can PUSH it back)

# The queue – abstract data type

In computer science, a queue is an abstract data type that serves as a collection of elements, with two principal operations:

- ENQUEUE, which **adds** an element to the collection, and
- DEQUEUE, which **accesses** and **removes** the first element that was not yet removed.

The order in which elements come off a queue gives rise to its alternative name, **FIFO** (first in, first out).

The queue has a **head** and a **tail**. When an element is enqueued, it takes its place at the tail of the queue, just as a newly arriving customer takes a place at the end of the line. The element dequeued is always the one at the head of the queue, like the customer at the head of the line who has waited the longest.

See the drawing on the whiteboard!

# The queue – data structure

Queue properties:

- solid
- sequential
- dynamic
- continuously represented

## The queue – representation

There are 3 main possibility for continuous queue representation:

- standard queue – not effective
- walking queue – better (especially with more memory)
- circular queue

Standard queue and walking queue representations will be explained in practical class.

# The queue – representation

There are 3 main possibility for continuous queue representation:

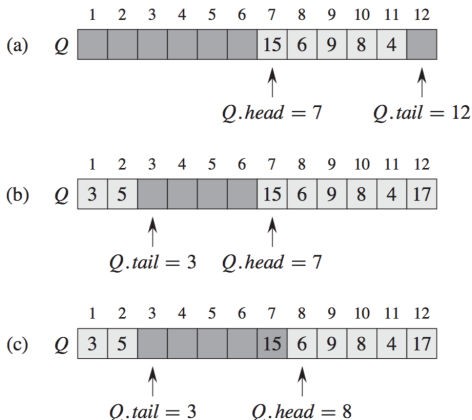
- standard queue – not effective
- walking queue – better (especially with more memory)
- **circular queue**

Let  $Q$  be a queue containing numbers. We can implement a circular queue of at most  $n-1$  elements with an array  $Q[1, \dots, n]$ .

The array has attributes  **$Q.head$**  and  **$Q.tail$** .

- When  $Q.head = Q.tail$ , the queue is empty.
- Initially, we have  $Q.head = Q.tail = 1$ .

# The queue – representation



**Figure 10.2** A queue implemented using an array  $Q[1..12]$ . Queue elements appear only in the lightly shaded positions. (a) The queue has 5 elements, in locations  $Q[7..11]$ . (b) The configuration of the queue after the calls  $ENQUEUE(Q, 17)$ ,  $ENQUEUE(Q, 3)$ , and  $ENQUEUE(Q, 5)$ . (c) The configuration of the queue after the call  $DEQUEUE(Q)$  returns the key value 15 formerly at the head of the queue. The new head has key 6.

## The queue – representation

- If we attempt to dequeue an element from an empty queue, the queue **underflows**.
- When  $Q.head = Q.tail + 1$ , the queue is full, and if we attempt to enqueue an element, then the queue **overflows**.

In our procedures `ENQUEUE` and `DEQUEUE`, we have omitted the error checking for underflow and overflow.

## The queue – operations

In our procedures ENQUEUE and DEQUEUE, we have omitted the error checking for underflow and overflow.

ENQUEUE( $Q, x$ )

```
1   $Q[Q.tail] = x$   
2  if  $Q.tail == Q.length$   
3       $Q.tail = 1$   
4  else  $Q.tail = Q.tail + 1$ 
```

DEQUEUE( $Q$ )

```
1   $x = Q[Q.head]$   
2  if  $Q.head == Q.length$   
3       $Q.head = 1$   
4  else  $Q.head = Q.head + 1$   
5  return  $x$ 
```

# The queue – data structure operations

## Implemented data structure operations:

- **create** data structure – continuous (vector) representation
- **modify** data structure
  - add elements – `ENQUEUE`
  - delete elements – `DEQUEUE`
  - change elements – **not possible**
- **access** – `DEQUEUE`

# The linked list – abstract data type

In computer science, a linked list is a linear collection of data elements, whose order is not given by their physical placement in memory. Instead, each element points to the next. Its complex elements contain

- **value**, and
- **pointer**, which stores a memory address of another element.

An attribute **head** points to the first element of the list.  
If head=NIL, the list is empty.

# The linked list – data structure

Linked list properties:

- solid
- sequential
- dynamic
- represented by pointers (linked representation)

## Possible linked lists

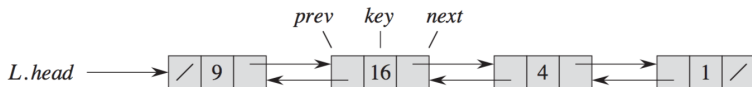
There are many possibility for linked lists:

- standard linked list
- circular linked list
- doubly linked list
- multilists

See the drawings on the whiteboard!

## Doubly linked list – details

- Each element of a doubly linked list  $L$  is an object with an attribute `key` and two other pointer attributes: `next` and `prev`.
- Given an element  $x$  in the list,  $x.next$  points to its successor in the linked list, and  $x.prev$  points to its predecessor.
- An attribute  $L.head$  points to the first element of the list.
- If  $x.prev = \text{NIL}$ , the element  $x$  has no predecessor and is therefore the first element. If  $x.next = \text{NIL}$ , the element  $x$  has no successor and is therefore the last element.



## Doubly linked list – operations

The procedure LIST-SEARCH( $L, k$ ) finds the first element with key  $k$  in list  $L$  by a simple linear search, returning a pointer to this element. If no object with key  $k$  appears in the list, then the procedure returns NIL.

LIST-SEARCH( $L, k$ )

```
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

## Doubly linked list – operations

Given an element  $x$  whose key attribute has already been set, the LIST-INSERT procedure "splices"  $x$  onto the front of the linked list.

LIST-INSERT( $L, x$ )

- 1  $x.next = L.head$
- 2 **if**  $L.head \neq \text{NIL}$
- 3      $L.head.prev = x$
- 4  $L.head = x$
- 5  $x.prev = \text{NIL}$

## Doubly linked list – operations

The procedure `LIST-DELETE` removes an element  $x$  from a linked list  $L$ . It must be given a pointer to  $x$ , and it then "splices"  $x$  out of the list by updating pointers. If we wish to delete an element with a given key, we must first call `LIST-SEARCH` to retrieve a pointer to the element.

`LIST-DELETE( $L, x$ )`

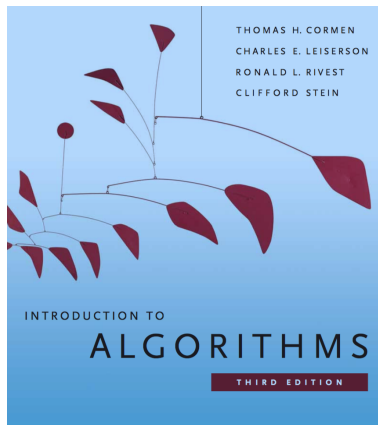
```
1  if  $x.prev \neq \text{NIL}$ 
2       $x.prev.next = x.next$ 
3  else  $L.head = x.next$ 
4  if  $x.next \neq \text{NIL}$ 
5       $x.next.prev = x.prev$ 
```

# Doubly linked list – data structure operations

## Implemented data structure operations:

- **create** data structure – **linked** representation
- **modify** data structure
  - add elements – LIST-INSERT
  - delete elements – LIST-DELETE
  - change elements – **not possible in this representation, but possible in some other representations**
- **access** – LIST-SEARCH

# References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.